

A combinational circuit is a circuit whose outputs depend only on the present values of its inputs. There is no memory and no feedback, give it the same inputs, and you always get the same outputs.

The design always begins with a **specification** (a description of the desired behavior) and ends with either a **logic diagram** or a set of **Boolean functions** from which the diagram can be drawn.

The four steps of Design Procedure:

1. Identify inputs and outputs. From the specification, determine how many inputs and outputs are needed, and give each a symbol (a letter).
2. Build the truth table. Write down the exact output value for every possible combination of inputs.
3. Simplify each output. Find the simplest Boolean expression for each output (using algebra/K-maps).
4. Draw the logic diagram: check that it is correct.

Example: BCD-to-Excess-3 Code Converter

A **code converter** translates information from one binary code into another so that two systems using different codes can talk to each other.

Step 1: inputs/outputs

Each code uses 4 bits to represent one decimal digit, so we need 4 inputs (A, B, C, D) and 4 outputs (w, x, y, z).

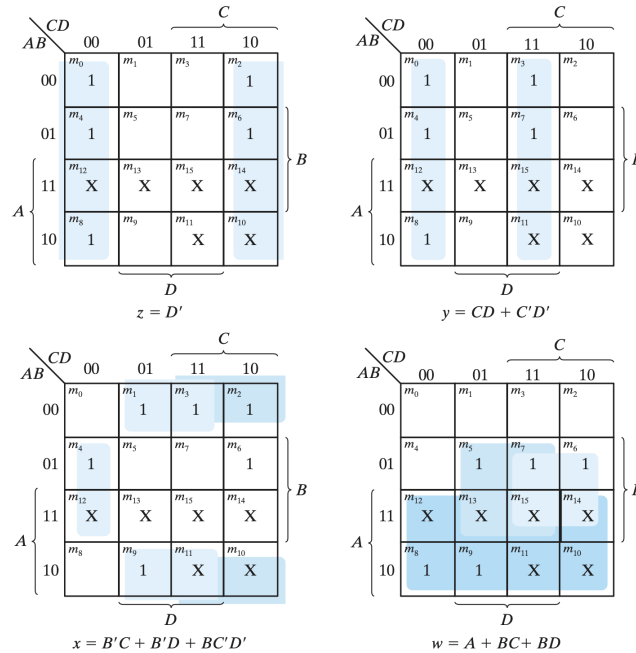
Step 2: truth table

BCD only uses the combinations for decimal digits 0 through 9. The Excess-3 code for a digit is simply that digit's value plus 3, written in binary.

Decimal digit	Input BCD				Output Excess-3			
	A	B	C	D	w	x	y	z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

Our bits give $2^4 = 16$ possible combinations, but only 10 are valid decimal digits. The six leftover combinations (binary 10 through 15) never occur in valid BCD. These are our don't-cares. If we do not use them, our simplified expressions will form very complex digital circuits and it will prevent a proper connection from occurring across all the outputs.

Step 3: simplify



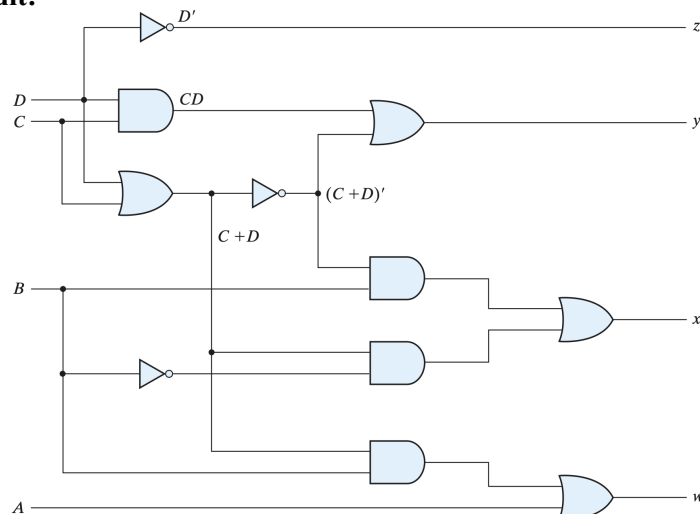
Plotting each output on a 4-variable K-map (using the don't-cares to our advantage) gives the simplified sum-of-products expressions:

$$\begin{aligned}
 z &= D' \\
 y &= CD + C'D' \\
 x &= B'C + B'D + BC'D' \\
 w &= A + BC + BD
 \end{aligned}$$

These can be algebraically manipulated so that outputs share a gate (such as $C + D$) in this instance:

$$\begin{aligned}
 z &= D' \\
 y &= CD + (C + D)' \\
 x &= B'(C + D) + BC'D' \\
 w &= A + B(C + D)
 \end{aligned}$$

Step 4: draw the circuit:



Arithmetic is the most fundamental thing a computer does, and addition is the most basic arithmetic operation. An adder is created in a hierarchy:

Half adder → full adder → multi-bit adders

The four possible single-bit additions are:

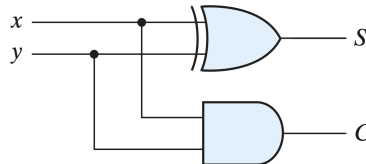
$0 + 0 = 0$ $0 + 1 = 1$ $1 + 0 = 1$ $1 + 1 = 10$

A **half adder** adds two single bits. It has two inputs (x and y) and two outputs: S (sum) and C (carry).

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The carry C is 1 only when both inputs are 1 (that is an AND). The sum S is 1 when exactly one input is 1 (that is an XOR). The simplified expressions are:

$$S = x'y + xy' = x \oplus y, \quad C = xy$$



A half adder cannot accept a carry coming in from a previous stage. It only adds two bits. To add a carry-in as well, we need a full adder. A **full adder** adds three bits: the two bits being added (x and y) plus a carry-in z from the previous, lower position. It still produces two outputs, S (sum) and C (carry-out).

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

To get the boolean expression, we can write the expression in minterm POS form. Our simplified expressions for a full adder is:

$$S = z \oplus (x \oplus y) \quad C = xy + xz + yz$$

